University of Exeter

ECM2433: The C Family

Lecture 08: Closures and Iterators

Why closures and iterators?

```rust
// What you write today
let mut results = Vec::new();
for line in &lines {                          // [1] iterate
    let trimmed = line.trim();                // [2] transform
    if !trimmed.is_empty() {                  // [3] filter
        results.push(trimmed.to_string());    // [4] collect
    }
}

// What you'll write after this video
let results: Vec<String> = lines.iter()       // [1] iterate
    .map(|line| line.trim())                  // [2] transform
    .filter(|line| !line.is_empty())          // [3] filter
    .map(|line| line.to_string())
    .collect();                               // [4] collect
```

- Both do the same thing: trim lines and keep non-empty ones
- Iterator chains replace manual loops with composable steps
- Closures (`|line| ...`) are the glue that makes this work
- [Book ch 13.1, 13.2]

University of Exeter

Closure syntax and basics

```rust
fn add_fn(a: i32, b: i32) -> i32 {     // regular function
    a + b
}

let add = |a: i32, b: i32| -> i32 { a + b };  // closure (explicit types)
let double = |x| x * 2;                       // closure (inferred types)

println!("{}", add(2, 3));     // 5
println!("{}", double(4));     // 8
```

- Closures: anonymous functions defined with `|params| body`
- Type annotations are usually optional (compiler infers from usage)
- Single expressions do not need braces
- Call closures like regular functions
- Like C++ lambdas, but with simpler syntax

University of Exeter

- [Book ch 13.1]

Capturing variables

```rust
// Example 1: borrows by reference, read-only
let name = String::from("Alice");
let greeting = || println!("Hello, {name}!");
greeting(); // Hello, Alice!

// Example 2: borrows by reference, mutates
let mut count = 0;
let mut increment = || count += 1;
increment();
increment();
println!("{count}"); // 2

// Example 3: move ownership in, but only reads
let name = String::from("Bob");
let greet = move || println!("Hello, {name}!");
greet();
println!("{name}"); // will not compile: name was moved
```

- Closures capture variables from their enclosing scope
- By default: borrows (shared or mutable, as needed)
- `move` keyword: captures by value (moves for non-`Copy` values)
- Compiler chooses the least restrictive capture automatically
- [Book ch 13.1]

Fn, FnMut, and FnOnce

Trait	Call requirement	Meaning	Example
`Fn`	`&self`	Callable repeatedly; does not mutate or move out of captures	`|x| x + 1`
`FnMut`	`&mut self`	Callable repeatedly; may mutate captured state	`|| count += 1`
`FnOnce`	`self`	May consume captures; callable at least once	`|| v.push(name)`

- Every closure implements `FnOnce` (can always be called at least once)
- Closures that do not move out of captures can also implement `FnMut`
- Closures that also avoid mutating captures can also implement `Fn`
- `move` changes capture mode, but does not automatically make a closure `FnOnce`-only
- Most iterator adaptors require `FnMut`
- You rarely write these bounds yourself; the compiler guides you
- [Book ch 13.1]

Creating iterators and the Iterator trait

```rust
let v = vec![10, 20, 30];
let mut iter = v.iter();        // creates an iterator

println!("{:?}", iter.next()); // Some(&10)
println!("{:?}", iter.next()); // Some(&20)
println!("{:?}", iter.next()); // Some(&30)
println!("{:?}", iter.next()); // None
```

```rust
trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
    // ... many provided methods (map, filter, etc.)
}
```

- `.iter()` creates an iterator over references (`&T`)
- `next()` returns `Some(item)` or `None` when exhausted
- `for x in &v` is shorthand: it calls `.iter()` and `next()` for you
- Iterators are lazy: they do nothing until consumed
- [Book ch 13.2]

University of Exeter

Iterator adaptors: map and filter

```rust
let v = vec![1, 2, 3, 4, 5];

let doubled: Vec<i32> = v.iter()
    .map(|x| x * 2)
    .collect();                // [2, 4, 6, 8, 10]

let evens: Vec<&i32> = v.iter()
    .filter(|x| *x % 2 == 0)
    .collect();                // [2, 4]

// fully type-hinted and dereferenced versions
let doubled: Vec<i32> = v.iter().map(|x: &i32| *x * 2).collect();
let evens: Vec<&i32> = v.iter().filter(|x: &&i32| **x % 2 == 0).collect();
```

- `map`: transforms each element using a closure
- `filter`: keeps elements where the closure returns `true`
- Both return new iterators (lazy, no work done yet)
- `.collect()` drives the chain and produces a result
- [Book ch 13.2]

Iterator adaptors: enumerate and zip

```rust
let words = vec!["hello", "world", "rust"];

// enumerate: pairs each element with its index
for (i, word) in words.iter().enumerate() {
    println!("{i}: {word}");
}
// 0: hello
// 1: world
// 2: rust

let keys = vec!["a", "b", "c"];
let vals = vec![1, 2, 3];

// zip: pairs elements from two iterators
let pairs: Vec<(&&str, &i32)> = keys.iter()
    .zip(vals.iter())
    .collect();   // [("a", 1), ("b", 2), ("c", 3)]
```

- `enumerate`: yields `(index, element)` tuples
- `zip`: combines two iterators into pairs, stops at the shorter one
- Both are adaptors: lazy until consumed
- [Book ch 13.2]

University of Exeter

Consuming iterators: collect and type hints

```rust
let v = vec![1, 2, 3, 4, 5];

// Type annotation on the variable
let doubled: Vec<i32> = v.iter()
    .map(|x| x * 2)
    .collect();

// Turbofish syntax on collect
let doubled = v.iter()
    .map(|x| x * 2)
    .collect::<Vec<i32>>();
```

- `collect` transforms an iterator into a collection
- Rust must know the target collection type
- Type on the `let` binding or turbofish (`::<>`) on `collect`
- Can collect into `Vec`, `HashMap`, `String`, and more
- [Book ch 13.2]

University of Exeter

Consuming iterators: sum, fold and count

```rust
let v = vec![1, 2, 3, 4, 5];

let total: i32 = v.iter().sum();                          // 15
let total = v.iter().fold(0, |acc, x| acc + x);           // 15
let product = v.iter().fold(1, |acc, x| acc * x);         // 120
let max = v.iter().fold(i32::MIN, |acc, x| {              // 5
    if *x > acc { *x } else { acc }
});
let num_evens = v.iter().filter(|x| *x % 2 == 0).count(); // 2
```

- `sum`: adds up all elements (needs a type hint)
- `fold`:
  - reduces an iterator to a single value using an accumulator
  - takes an initial value and a closure `|accumulator, element|`
  - More flexible than `sum`: works for any reduction operation
- `count`: counts the number of elements
- [Book ch 13.2]

Implementing Iterator for a custom type

```rust
struct Fibonacci {
    a: u64,
    b: u64,
}

impl Fibonacci {
    fn new() -> Self {
        Fibonacci { a: 0, b: 1 }
    }
}

impl Iterator for Fibonacci {
    type Item = u64;

    fn next(&mut self) -> Option<Self::Item> {
        let value = self.a;
        (self.a, self.b) = (self.b, self.a + self.b);
        Some(value)
    }
}

let fibs: Vec<u64> = Fibonacci::new()
    .take(8)
    .collect();          // [0, 1, 1, 2, 3, 5, 8, 13]
```

- Define a struct to hold iterator state
- Implement `Iterator` with `type Item` and `next()`
- Return `Some(value)` to yield, `None` to stop (`Some` forever for infinite iterators)
- `.take(n)`: adaptor that limits an infinite iterator to n elements
- All adaptor/consumer methods become available automatically
- [Book ch 13.2]

Chaining adaptors

```rust
let words = vec!["hello", "  ", "world", "", "rust", "  hi  "];

let result: Vec<String> = words.iter()
    .map(|w| w.trim())
    .filter(|w| !w.is_empty())
    .enumerate()
    .map(|(i, w)| format!("{}: {}", i + 1, w))
    .collect(); // ["1: hello", "2: world", "3: rust", "4: hi"]
```

- Adaptors chain: each one returns a new iterator
- Read top to bottom: trim, filter empties, number, format
- The whole chain is lazy until `.collect()` drives it
- No intermediate `Vec`s are created: elements flow through one at a time
- [Book ch 13.2]

University of Exeter

Demo

University of Exeter

Recap

- Closures: `|params| body`, automatic capture, `move`, and `Fn`/`FnMut`/`FnOnce`
- Iterators: `.iter()` creates one, `next()` yields `Some` or `None`
- Adaptors: `map`, `filter`, `enumerate`, `zip`, `take` (lazy, return new iterators)
- Consumers: `collect`, `sum`, `fold`, `count` (drive the chain, produce a result)
- Custom iterators: implement `Iterator` trait with `type Item` and `next()`
- Required reading: Book ch 13.1 (closures), 13.2 (iterators)

University of Exeter